

# ハンバーガーメニュー内項目の動的制御

表示されている画面やユーザーの権限に応じて動的に制御する機能の実装に取り掛かりましょう。

## 理想的なメニュー表示制御の考察

Vue Router環境において、メニュー項目を「ダッシュボード画面だからこれ」「打刻画面だからこれ」と画面ごとに愚直に切り替えるのは、将来画面が増えたときに管理が非常に煩雑になります。

そこで、現在のログイン状態とユーザーの権限（`isAuth`）をベースに、以下のようなルールで**メニュー項目を動的に一元管理（フィルタリング）する設計**が最もスマートで拡張性があります。

## メニューに表示すべき項目の整理

- ログインしている全ユーザー（一般・管理者共通）にいつでも出す項目
  - ダッシュボード
  - 打刻画面
  - 予定申請
  - 打刻内容編集申請
  - アカウント情報編集
  - ログアウト
- 管理者ユーザー（ `isAuth === 1` ）だけに特別に出す項目
  - 管理者画面（※将来の拡張用フラグ）

現在の画面がどれであっても、ハンバーガーメニューを開いたときには上記の「今使える機能」がフルラインナップで綺麗に並んでいる状態が、WebアプリケーションのUI/UX（操作性）として最も自然で使いやすくなります。

## 修正対象：AppHeader.vue の実装案

ハンバーガーメニューの内部ロジックを、ローカルストレージから取得したユーザー情報（特に名前や権限 `isAuth`）に基づいて動的に生成するように書き換えます。

以下に、実用的な `AppHeader.vue` のソースコード案をご提示します。

```
<template>
  <header class="app-header">
    <div class="header-brand" @click="goHome">
      <span class="brand-title">勤怠管理システム</span>
    </div>

    <button class="menu-trigger" @click="isMenuOpen = !isMenuOpen" :class="{ 'is-active': isMenuOpen }">
      <span></span><span></span><span></span>
    </button>
    <transition name="slide">
      <nav v-if="isMenuOpen" class="nav-menu">
        <div class="menu-user-info">
          <p class="user-name">👤 {{ loginUserName }} さん</p>
          <span v-if="userRole === 1" class="badge-admin">管理者</span>
        </div>
      </nav>
    </transition>
  </header>
</template>
```

```

    <ul class="menu-list">
      <li v-for="item in menuItems" :key="item.path" class="menu-item">
        <button @click="navigate(item.path, item.action)" :class="{ 'btn-logout': item.action === 'logout' }">
          {{ item.label }}
        </button>
      </li>
    </ul>
  </nav>
</transition>
</header>
</template>

<script setup>
import { ref, computed, onMounted, watch } from 'vue';
import { useRouter, useRoute } from 'vue-router';

const router = useRouter();
const route = useRoute();

const isMenuOpen = ref(false);
const loginUserName = ref('ゲスト');
const userRole = ref(0); // 0: 一般, 1: 管理者

// ローカルストレージからユーザー情報を読み込んで復元する関数
const loadUserInfo = () => {
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    loginUserName.value = user.userName;
    userRole.value = user.isAuth || 0; // 0:一般, 1:管理者
  }
};

```

```
onMounted(() => {
  loadUserInfo();
});

// ★ URL（ルート）が変わるたびにユーザー情報を再チェックする（画面遷移時の同期対策）
watch(() => route.path, () => {
  loadUserInfo();
  isMenuOpen.value = false; // 画面遷移したらメニューを自動で閉じる
});

// ★ 【核心】表示すべきメニュー項目をユーザーの権限に応じて動的に計算
const menuItems = computed(() => {
  // 全ユーザーに共通して表示するメニュー
  const items = [
    { label: 'ダッシュボード', path: '/dashboard' },
    { label: '打刻画面', path: '/attendance' },
    { label: '予定申請', path: '/schedule-request' }, // パスは既存の定義に合わせてください
    { label: '打刻内容編集申請', path: '/edit-request' }, // パスは既存の定義に合わせてください
    { label: 'アカウント情報編集', path: '/account-edit' },
  ];

  // ★ ユーザーが「管理者」の場合のみ、管理者専用画面のメニューを先頭または末尾に滑り込ませる
  if (userRole.value === 1) {
    items.push({ label: '🔧 管理者画面', path: '/admin' }); // 将来用
  }

  // 最後に「ログアウト」ボタンを無条件で末尾に追加
  items.push({ label: '🚪 ログアウト', path: '/login', action: 'logout' });

  return items;
});
```

```
// 画面遷移・アクションのハンドリング
const navigate = (path, action) => {
  if (action === 'logout') {
    // ログアウト処理
    if (confirm('ログアウトしますか?')) {
      localStorage.removeItem('user');
      router.push('/login');
    }
  } else {
    // 通常の画面遷移
    router.push(path);
  }
  isMenuOpen.value = false; // メニューを閉じる
};
```

```
const goHome = () => {
  router.push('/dashboard');
};
</script>
```

```
<style scoped>
/* ヘッダー全体のスタイル固定設定 */
.app-header {
  position: fixed;
  top: 0;
  left: 0;
  width: 100%;
  height: 60px;
  background-color: #343a40;
  color: white;
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 0 20px;
  box-sizing: border-box;
  z-index: 1000;
}
```

```
.header-brand {  
  cursor: pointer;  
  font-weight: bold;  
  font-size: 1.2rem;  
}  
  
/* ハンバーガーボタンのスタイリング */  
.menu-trigger {  
  background: none;  
  border: none;  
  width: 30px;  
  height: 24px;  
  position: relative;  
  cursor: pointer;  
}  
.menu-trigger span {  
  display: block;  
  position: absolute;  
  width: 100%;  
  height: 2px;  
  background-color: white;  
  left: 0;  
  transition: all 0.3s;  
}  
.menu-trigger span:nth-child(1) { top: 0; }  
.menu-trigger span:nth-child(2) { top: 11px; }  
.menu-trigger span:nth-child(3) { top: 22px; }
```

```
/* メニュー展開時の三本線の変形アニメーション */
.menu-trigger.is-active span:nth-child(1) { transform: translateY(11px) rotate(45deg); }
.menu-trigger.is-active span:nth-child(2) { opacity: 0; }
.menu-trigger.is-active span:nth-child(3) { transform: translateY(-11px) rotate(-45deg); }

/* ドロワーメニュー */
.nav-menu {
  position: fixed;
  top: 60px;
  right: 0;
  width: 260px;
  height: calc(100vh - 60px);
  background-color: #ffffff;
  box-shadow: -2px 0 8px rgba(0,0,0,0.15);
  padding: 20px;
  box-sizing: border-box;
}
.menu-user-info {
  border-bottom: 1px solid #dee2e6;
  padding-bottom: 15px;
  margin-bottom: 15px;
}
.user-name {
  color: #333;
  font-weight: bold;
  margin: 0 0 5px 0;
}
.badge-admin {
  background-color: #dc3545;
  color: white;
  font-size: 0.75rem;
  padding: 2px 6px;
  border-radius: 4px;
}
```



```
.menu-list {
  list-style: none;
  padding: 0;
  margin: 0;
}
.menu-item {
  margin-bottom: 10px;
}
.menu-item button {
  width: 100%;
  text-align: left;
  padding: 12px 15px;
  background: none;
  border: none;
  font-size: 1rem;
  color: #495057;
  cursor: pointer;
  border-radius: 6px;
  transition: background 0.2s;
}
.menu-item button:hover {
  background-color: #f8f9fa;
  color: #007bff;
}
.menu-item button.btn-logout {
  color: #dc3545;
  border-top: 1px solid #f1f3f5;
  margin-top: 10px;
  border-radius: 0;
}
.menu-item button.btn-logout:hover {
  background-color: #fff5f5;
}

/* スライドアニメーション */
.slide-enter-active, .slide-leave-active { transition: transform 0.3s ease; }
.slide-enter-from, .slide-leave-to { transform: translateX(100%); }
</style>
```

## 💡 この実装のポイントとメリット

### 1. `watch` による画面遷移との完全同期

`watch(() => route.path, ...)` を仕掛けたことで、他の画面から別画面へと移り変わった瞬間、メニュー側がローカルストレージを再読込します。これで「ログインし直したのにメニューの名前が古い」「権限が切り替わらない」といった同期ズレが完全に防げます。

### 2. `computed` による管理者画面（`isAuth === 1`）の動的制御

`userRole.value === 1` の条件を満たした時だけ、配列に「管理者画面」が自動プッシュされます。一般ユーザー（`isAuth === 0`）の画面では、項目自体が最初から配列に生成されないため、ソースコードを覗かれても不正遷移されるリスクを元からシャットアウトできます。

### 3. ログアウト処理の一体化

ログアウトアクションを組み込み、ストレージのクリア（`localStorage.removeItem('user')`）と `/login` へのリダイレクトを一連の挙動として統合しました。

このコードを現在の `AppHeader.vue` と差し替えるか、ロジックを統合してみてください。